NFXP Reference and Known-Truth Validation Tutorial

Nested Fixed Point Estimation for Dynamic Discrete Choice

econirl package documentation

May 2026

RTD page: https://econirl.readthedocs.io/en/latest/estimators/nfxp.html **Source branch:** https://github.com/rawatpranjal/EconIRL/tree/feature/known-truth-dgp-harness

1 Purpose

NFXP, the nested fixed point estimator, is the reference estimator for structural dynamic discrete choice. It estimates primitive reward parameters by solving the agent's dynamic programming problem inside the likelihood. The method is the baseline against which CCP, MPEC, NNES, TD-CCP, SEES, and IRL-style estimators should be compared.

This document is a known-truth validation tutorial. It does not use real data. The synthetic DGP supplies the true reward, transition law, value function, policy, and counterfactual oracle objects. That lets us ask a sharp question: when the NFXP assumptions hold, does the implementation recover the ground truth at medium scale?

2 Source Papers

The primary source is Rust (1987), which formalized the bus-engine replacement problem and introduced nested fixed point maximum likelihood. The computational upgrade is Iskhakov, Rust, Schjerning, and Seo (2016), which recommends a successive-approximation plus Newton-Kantorovich inner loop. The identification lesson that matters for this validation is standard in dynamic discrete choice: the reward scale and location must be normalized, transitions must be handled separately from payoffs, and reward features must vary enough across actions to identify the structural parameters.

3 When to Use NFXP

Use NFXP when the state space is tabular, the transition kernel is known or can be estimated in a first stage, the reward is parameterized by a low-dimensional vector, and exact dynamic programming is computationally feasible. NFXP is the right tool when the goal is publication-grade structural estimates, likelihood values, analytical scores, standard errors, and counterfactual policies from recovered primitives.

NFXP is usually the wrong tool for high-dimensional or continuous state spaces. For those cases, use NNES, SEES, GLADIUS, or TD-CCP depending on whether the goal is structural parameter recovery, value-function approximation, or transition-free estimation.

4 Model

The observed data are state-action-next-state trajectories

$$\{(s_{it}, a_{it}, s_{i,t+1}) : i = 1, \ldots, N,\ t = 1, \ldots, T\}.$$

The flow payoff is linear in known features:

$$u_\theta(s,a) = \phi(s,a)^\top \theta.$$

The transition kernel is

$$P_a(s,s') = \Pr(s_{t+1} = s' \mid s_t = s, a_t = a).$$

The discount factor is β and the i.i.d. type-I extreme-value shock scale is σ. The integrated value function solves the soft Bellman fixed point

$$V_\theta(s) = \sigma \log \sum_a \exp\left(\frac{u_\theta(s,a) + \beta \sum_{s'} P_a(s,s') V_\theta(s')}{\sigma}\right).$$

The choice-specific value is

$$Q_\theta(s,a) = u_\theta(s,a) + \beta \sum_{s'} P_a(s,s') V_\theta(s'),$$

and the implied conditional choice probability is

$$\pi_\theta(a \mid s) = \frac{\exp(Q_\theta(s,a)/\sigma)}{\sum_b \exp(Q_\theta(s,b)/\sigma)}.$$

NFXP maximizes the conditional log-likelihood

$$\hat{\theta} = \arg\max_\theta \sum_{i,t} \log \pi_\theta(a_{it} \mid s_{it}).$$

5 Estimator Mechanics

For each candidate θ, the inner loop solves $V = TV$. EconIRL supports successive approximation, Newton-Kantorovich, and a hybrid solver. The default known-truth validation uses the hybrid solver: safe successive approximation far from the fixed point and Newton-Kantorovich updates near the solution.

The analytical score differentiates through the fixed point. For observation i, the score has the logit form

$$\psi_i(\theta) = \frac{1}{\sigma}\left[\nabla_\theta Q(s_i,a_i) - \sum_a \pi_\theta(a \mid s_i) \nabla_\theta Q(s_i,a)\right],$$

with

$$\nabla_\theta Q(s,a) = \phi(s,a) + \beta \sum_{s'} P_a(s,s') \frac{\partial V(s')}{\partial \theta}.$$

The total derivative solves

$$(I - \beta P_\pi)\frac{\partial V}{\partial \theta} = \sum_a \pi_\theta(a \mid s)\phi(s,a).$$

The outer loop uses BHHH by default. It builds the positive semi-definite outer-product matrix from per-observation scores, applies damping when needed, and uses a line search to maintain likelihood improvement.

6 Identification Checks

The known-truth DGP must be identified before we judge the estimator. Rust-style NFXP handles this through three restrictions.

1. **Conditional independence.** The state transition is Markov in the observed state and action and does not depend on the current logit shock.

2. **Transition separation.** The transition process is not estimated inside the payoff likelihood in this tutorial. The synthetic DGP supplies the true transition tensor exactly.
3. **Reward normalization.** The shock scale is fixed at $\sigma = 1$. The synthetic DGP uses an exit action and absorbing state as a zero-reward anchor, so payoff location is pinned down.

The practical identification failure to watch for is action support. If one action is almost never chosen, the likelihood cannot pin down that action's payoff. The second failure is state-only reward variation copied across actions. That collapses action-specific payoff differences and makes a structural theta recovery test ill posed.

7 Known-Truth DGP

The canonical NFXP validation cell is `canonical_low_action`. It is small enough to solve exactly and large enough to expose rank, support, and normalization failures.

Table 1: Canonical NFXP DGP.

Quantity	Value
Regular states	20
Absorbing states	1
Total states	21
Actions	3
Exit action	2
Discount factor	0.95
Shock scale	1.0
Reward mode	action-dependent
Reward dimension	low
State mode	low-dimensional
Simulated individuals	2,000
Periods per individual	80
Observations	160,000

The action-dependent features are

$$\phi(s,a) = \begin{cases} (1, x_s, 0, 0) & a = 0, \\ (0, 0, 1, x_s) & a = 1, \\ (0, 0, 0, 0) & a = 2. \end{cases}$$

The exit action has zero reward and sends the process to the absorbing state. The true parameter vector is:

Table 2: True reward parameters.

Parameter	Truth
`action_0_intercept`	0.10
`action_0_progress`	0.50
`action_1_intercept`	0.00
`action_1_progress`	-0.20

8 Pre-Estimation Diagnostics

Run diagnostics before fitting. If these checks fail, the problem is usually the DGP or feature design, not the optimizer.

```
from experiments.known_truth import (
    SimulationConfig,
```

```
    build_known_truth_dgp,
    get_cell,
    run_pre_estimation_diagnostics,
    simulate_known_truth_panel,
)

cell = get_cell("canonical_low_action")
dgp = build_known_truth_dgp(cell.dgp_config)
panel = simulate_known_truth_panel(
    dgp,
    SimulationConfig(n_individuals=2000, n_periods=80, seed=42),
)
diagnostics = run_pre_estimation_diagnostics(dgp, panel)

assert diagnostics.passed
assert diagnostics.feature_rank == diagnostics.num_features
assert diagnostics.is_action_dependent
```

Table 3: Pre-estimation checks from the medium-scale run.

Check	Value	Status
Feature rank	4 / 4	pass
Feature condition number	4.512	pass
Transition row error	2.42×10^{-8}	pass
Observed states	21 / 21	pass
State-action coverage	1.000	pass
Action shares	0.345, 0.330, 0.325	pass
Minimum action share	0.325	pass
Exit/absorbing anchor	true	pass

9 Run the Tutorial

From the repository root:

```
PYTHONPATH=src:. python -m experiments.known_truth \
    --estimator NFXP \
    --cell-id canonical_low_action \
    --output-dir outputs/known_truth \
    --show-progress \
    --verbose
```

The command builds the DGP, solves the true Bellman problem, simulates the panel with `tqdm` progress, runs pre-estimation diagnostics, fits NFXP, evaluates reward/value/Q/policy recovery, solves Type A/B/C counterfactual oracles, writes one `result.json`, and raises if any non-smoke hard gate fails. NFXP itself does not need a GPU on this tabular cell, but the same module command is suitable for RunPod workers so the estimator validation interface is shared across methods.

Direct estimator usage:

```
from experiments.known_truth import (
    build_known_truth_dgp,
    get_cell,
    known_truth_initial_params,
    simulate_known_truth_panel,
)
from econirl.estimation.nfxp import NFXPEstimator

cell = get_cell("canonical_low_action")
```

```
dgp = build_known_truth_dgp(cell.dgp_config)
panel = simulate_known_truth_panel(dgp, cell.simulation_config)

estimator = NFXPEstimator(
    optimizer="BHHH",
    inner_solver="hybrid",
    inner_tol=1e-12,
    inner_max_iter=100_000,
    outer_max_iter=500,
    compute_hessian=True,
    verbose=True,
)

summary = estimator.estimate(
    panel=panel,
    utility=dgp.utility(),
    problem=dgp.problem,
    transitions=dgp.transitions,
    initial_params=known_truth_initial_params(dgp),
)
```

10 Estimation Results

The medium-scale validation run used 2,000 individuals, 80 periods per individual, and 160,000 observations.

Table 4: Run summary.

Quantity	Value
Converged	true
Outer iterations	12
Log-likelihood	-174875.7719
Estimation time	5.28 seconds
Function evaluations	80
Total inner iterations	11259
Final inner iterations	139
Inner solver	hybrid
Outer optimizer	BHHH

Table 5: Parameter recovery.

Parameter	Truth	Estimate	SE	Error
`action_0_intercept`	0.100000	0.083894	0.029335	-0.016106
`action_0_progress`	0.500000	0.528522	0.035889	0.028522
`action_1_intercept`	0.000000	-0.014461	0.036733	-0.014461
`action_1_progress`	-0.200000	-0.200511	0.052502	-0.000511

Table 6: Recovery metrics.

Metric	Value
Parameter RMSE	0.017904
Parameter relative RMSE	0.065378
Parameter cosine similarity	0.998867
Reward RMSE	0.009694
Value RMSE	0.019445
Q RMSE	0.022438
Policy KL	9.21×10^{-5}
Policy total variation	0.005697
Policy max state L1	0.018905

Table 7: Hard gates.

Gate	Threshold	Value	Status
Converged	true	true	pass
Parameter cosine	≥ 0.98	0.998867	pass
Parameter relative RMSE	≤ 0.15	0.065378	pass
Policy TV	≤ 0.03	0.005697	pass
Value RMSE	≤ 0.10	0.019445	pass

The estimates are not equal to truth because the panel is finite. The validation target is recovery within strict tolerances, not exact equality in one finite sample.

11 Counterfactuals

The tutorial evaluates three post-estimation counterfactuals.

Table 8: Counterfactual types.

Type	Intervention	Purpose
Type A	Shift reward, hold transitions fixed	Payoff counterfactual
Type B	Change transitions, hold reward fixed	State-dynamics counterfactual
Type C	Disable one non-anchor action	Action-set/design counterfactual

For each intervention, the harness solves the true counterfactual oracle. It then solves the same intervention using the reward recovered by NFXP and evaluates the resulting policy in the true counterfactual environment.

Table 9: Counterfactual recovery.

Counterfactual	Policy TV	Policy KL	Value RMSE	Regret
Type A	0.005109	7.56×10^{-5}	0.000238	0.000213
Type B	0.005457	8.20×10^{-5}	0.000363	0.000362
Type C	0.003548	3.56×10^{-5}	0.000114	0.000086

These regrets are small because the recovered reward is close enough to the true reward that re-solving the intervened model produces almost the same policy as the oracle.

12 Debugging Gaps

If NFXP fails to recover truth, debug in this order.

1. **Feature rank.** If rank is below the number of reward parameters, θ is not identified.
2. **Action support.** If one action is rarely observed, its payoff is weakly identified.
3. **Normalization.** If the exit/absorbing anchor is invalid, reward levels can drift.
4. **State-only rewards.** The canonical NFXP theta validation needs action-dependent features.
5. **Transition tensor.** NFXP needs stochastic transition rows and the right action-state-next-state orientation.
6. **Inner tolerance.** Use a tight tolerance such as 10^{-10} to 10^{-12} for structural validation.
7. **High beta.** Use `inner_solver="hybrid"` near $\beta = 1$; pure value iteration is intentionally slow there.
8. **Soft gates.** Do not accept a non-smoke run that only warns. Known-truth validation should pass hard gates or raise.

13 Practical Defaults

Recommended non-smoke validation settings:

```
NFXPEstimator(
    optimizer="BHHH",
    inner_solver="hybrid",
    inner_tol=1e-12,
    inner_max_iter=100_000,
    switch_tol=1e-3,
    outer_tol=1e-6,
    outer_max_iter=500,
    compute_hessian=True,
)
```

Use smoke settings only for code-path tests. A smoke test shows that the estimator runs and returns finite objects; it is not evidence of recovery.

14 Implementation Paths

- Estimator: `src/econirl/estimation/nfxp.py`
- Known-truth harness: `experiments/known_truth.py`
- Fast validation tests: `tests/test_known_truth.py`
- Master plan: `papers/econirl_package_jss/plans/README.md`
- NFXP alignment audit: `papers/econirl_package_jss/plans/alignment/02_nfxp.md`

15 References

Rust, J. (1987). Optimal Replacement of GMC Bus Engines: An Empirical Model of Harold Zurcher. *Econometrica*, 55(5), 999–1033.

Iskhakov, F., Lee, J., Rust, J., Schjerning, B., and Seo, K. (2016). Comment on "Constrained Optimization Approaches to Estimation of Structural Models." *Econometrica*, 84(1), 365–370.